

Agentraxis

Project Proposal

Project Advisor:

Ms. Sarah Abid Khan

Group Members:

Saim Arif Khan	23L-3071
Abdul Muiz	23L-3097
Jabeen Zahra	23L-3065

National University Of Computer and Emerging Sciences
Department of Computer Science
Lahore, Pakistan

Abstract

Agentraxis is a controlled educational platform for simulating selected AI-agent security risks and evaluating defensive controls. Through a web dashboard, a tester selects one of three predefined scenarios, prompt injection through a synthetic document, unauthorized access to a fictional data record, and unsafe request propagation between two simulated agents, and runs it against simulated agents and mock tools in either a vulnerable baseline mode or a protected mode. In protected mode, authorization is enforced at the backend tool boundary and does not rely on an agent's own claim that an action is safe. Every allowed and denied action is recorded in an audit log, and the outcomes of the two modes are compared using metrics calculated from actual stored runs. All demonstrations use synthetic data and mock tools, and the core system does not require a paid AI API.

1. Introduction

As AI agents are increasingly given access to tools, data, and other agents, they introduce security risks that differ from traditional software vulnerabilities like prompt injection, excessive permissions, and unsafe delegation between agents being among the most pressing. Agentraxis is proposed as a controlled, educational platform that lets a tester simulate these risks in a safe sandbox and evaluate whether backend-enforced defensive controls can stop them. The project is being developed as an Information Security course project by a three-person student team, and this document serves as the agreement between the project team and evaluators about what the initial system is intended to do.

The name Agentraxis combines “agent” and “praxis”, the Greek word for putting theory into practice, reflecting the project's goal of taking AI-agent security out of theory and into hands-on practice: agents are tested in a safe sandbox, and defensive controls are applied and measured against realistic attack scenarios. Agentraxis is a new, standalone application; it is not intended to connect to company systems or manage real confidential information. A web frontend communicates with a backend API that runs scenarios, invokes simulated agents, applies policy checks to mock tool requests, and stores test runs and events in a local database.

2. Goals and Objectives

The objectives of Agentraxis are as follows:

- Demonstrate selected AI-agent security risks in a safe, repeatable sandbox.
- Implement backend authorization checks that do not rely on an agent's own claim that an action is safe.
- Record allowed and denied actions in an audit log.
- Compare the outcomes of vulnerable and protected configurations.
- Present understandable results through a web dashboard.
- Measure the prototype using a defined test set and report actual results.

3. Scope of the Project

Agentraxis will provide a web interface through which a tester can select a predefined attack scenario, select a test mode (vulnerable or protected), execute the scenario against simulated agents and mock tools, inspect the resulting security decisions, and compare recorded outcomes. The initial release will focus on three scenarios: prompt injection through a synthetic document, unauthorized access to a fictional data record, and unsafe request propagation between two simulated agents. All demonstrations will use synthetic data and

mock tools; the project does not require a paid AI API and will not connect to real systems or manage real confidential information.

4. Initial Study and Work Done so Far

Prior to this proposal, the team reviewed the OWASP Top 10 for Large Language Model Applications together with the IEEE and ISO/IEC/IEEE software-requirements standards to identify which AI-agent risks are both demonstrable in a classroom setting and distinct from conventional application vulnerabilities. This study narrowed the scope to three representative threats, summarized below along with the control each is intended to demonstrate:

- Prompt injection — a synthetic document contains hostile text that attempts to trigger a restricted action; the prototype demonstrates untrusted-content handling combined with independent tool authorization.
- Excessive permissions — an agent may attempt to read data beyond its role; the prototype demonstrates least privilege and resource-level permission checks.
- Unsafe delegation — a receiving agent may trust an unapproved forwarded request; the prototype demonstrates independent authorization at each tool boundary.
-

Based on this study, the team has drafted the Software Requirements Specification for Agentraxis, defined functional, non-functional, and security requirements (including deny-by-default and fail-closed authorization at the backend tool boundary), and laid out an eight-phase development schedule covering requirements and setup, mock data and agents, the security engine, scenario implementation, persistence and API, the frontend, evaluation, and submission.

References

[1] IEEE, “IEEE Recommended Practice for Software Requirements Specifications,” IEEE Std 830-1998, 1998.

[2] ISO/IEC/IEEE, “Systems and software engineering – Life cycle processes – Requirements engineering,” ISO/IEC/IEEE 29148:2018, 2018.

[3] OWASP Foundation, “OWASP Top 10 for Large Language Model Applications.” [Online].Available: https://owasp.org/www-project-top-10-for-large-language-model-applications/